Agenda

- Remaining Types of Caching
  - Local Caching
  - Global Caching
- Cache eviction
- Cache invalidation.

=.

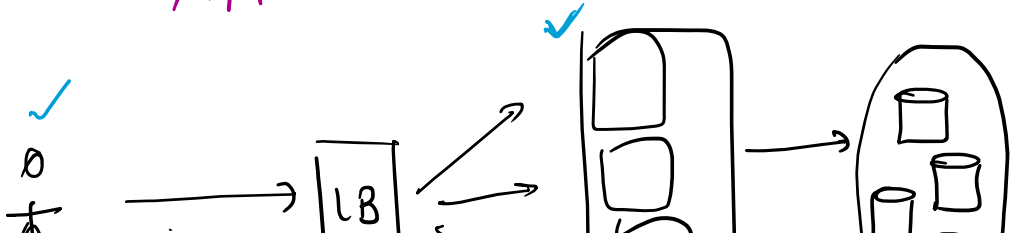
Complete SQL-Internals

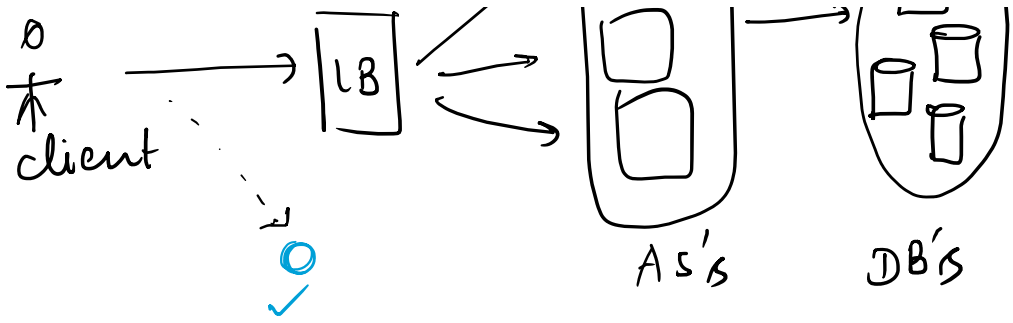
Resume HLD from CAP Theorem.

//

⊛ Local Caching

Caching some data on the Appl^n Server. (AS)

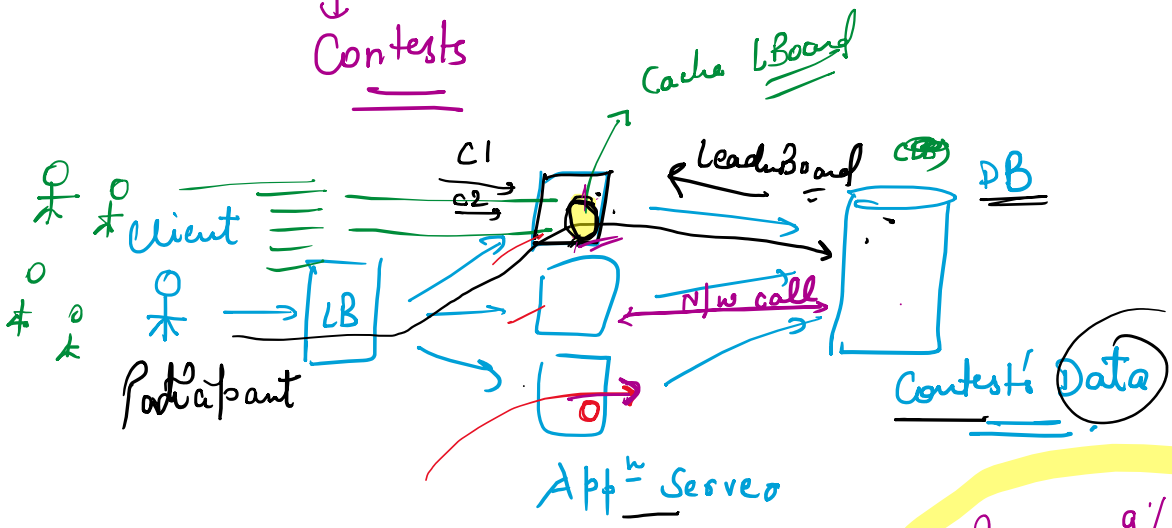




Scaler / AirTribe / UpGrad / Byjus / ——— .

↓

Contests



In 1 sec 10^9 / 10^8 ops =.

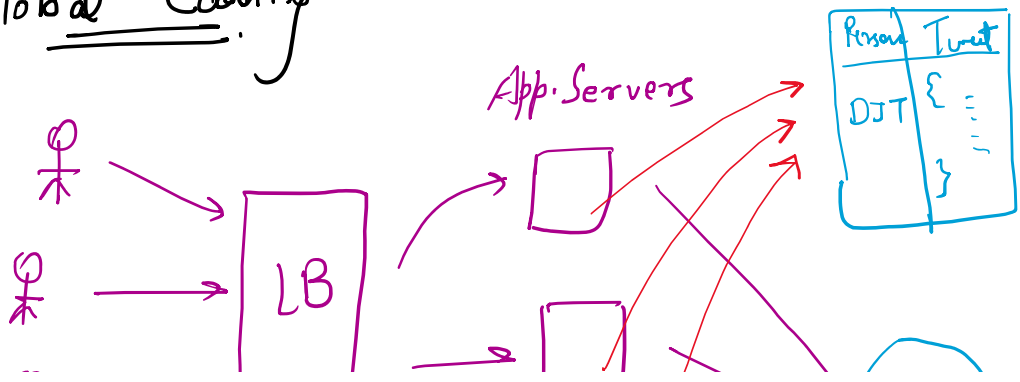
Fastest
- Memory ops → 1 sec = ~10^8
- (I/O) ops → ~10^5
- slowest N/w op → ~10^2

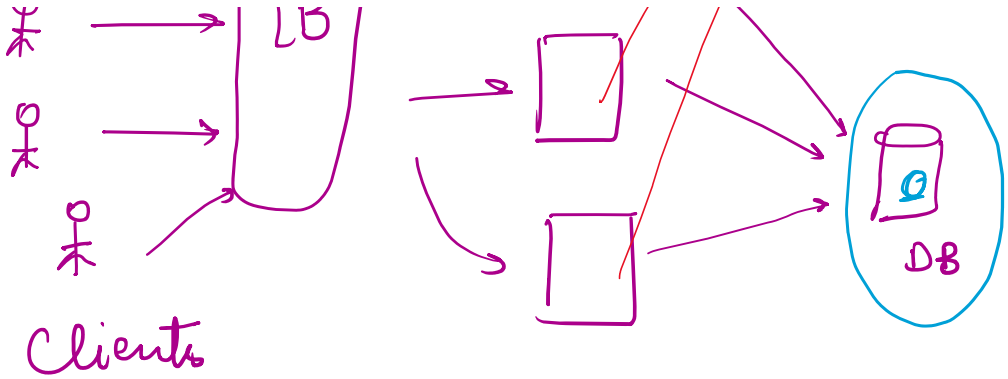
Optimize

1.) N/W overhead
2.) Disk Fetch

④ Global Caching

In-mem Storage aka Redis





After I've computed my res. from DB, I store that in another m/c.

Note:- NW overhead still remains intact.

reading the same data from a disk is ~80 times slower when same data read from an in-mem storage.

⊛ Prob. regarding Caching.

"Cache is never the orig. source of data"

→ COPY of the org. source.

Cache Invalidation

1.) copy (cache) & orig. source

Cache

1.) copy (cache) & org. source may get out of sync.

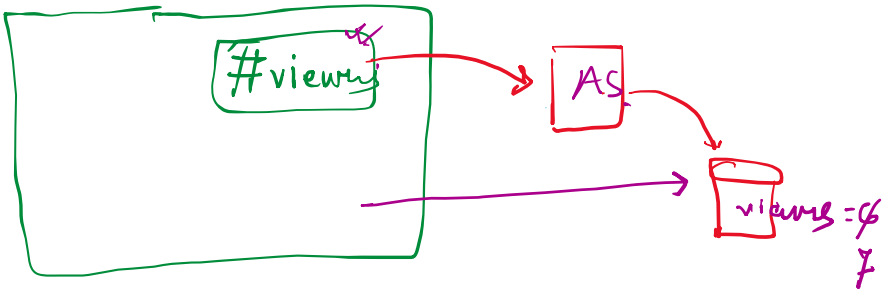
2.) Space in HDD (SSD) >>> Space in RAM @ same cost.

Cache eviction

⑧ How to keep Original & the Copy of Data in sync?

① TTL : Time to Live

Build live_view counter of an IPL Match



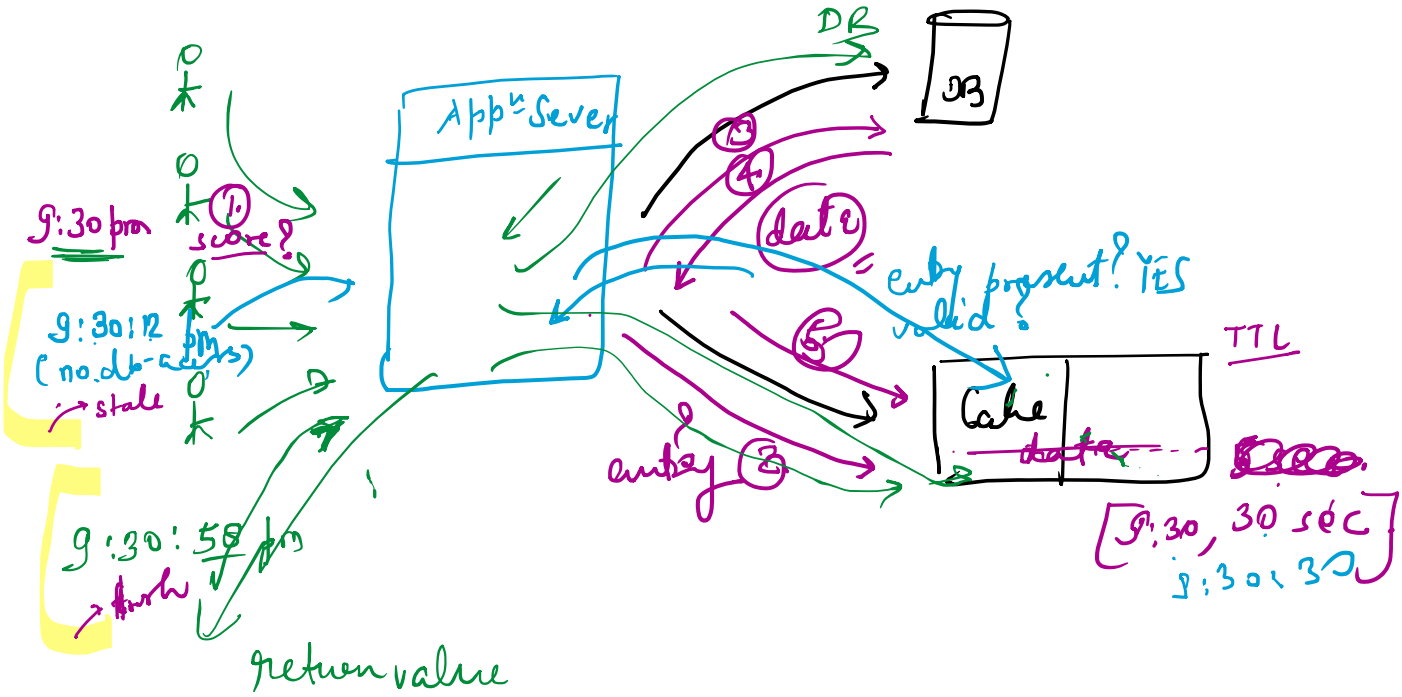
If we're okay for data to be non-exact for some amount of time

LIVE

last known.

Whenever I store data in cache, I set a TTL along with it.

after that amount of time, data will get auto-deleted.



If TTL is 1 sec.
& Load on DB is 1 MRPS

⇒ every 1 sec you make a call to DB for invalidation

Read

1.) Is the data in cache (& TTL not expired)

- YES — Fetch data from cache & return
- NO — (Never existed OR exists but TTL expired) → fetch data from DB
  - Store it in the cache along w/ TTL
  - Return data

Write (CUD)

Update the DB

Note :-

If you ever have a situation where you need to reduce the load on DB, then caching

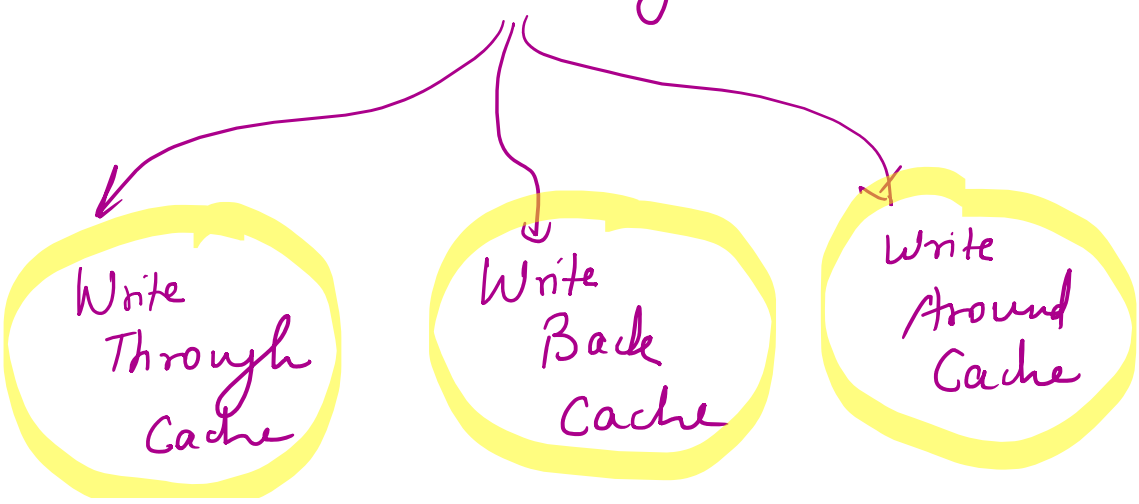
Read on DB,
try caching.

↳ TTL approach can only be used if you're okay w/ a bit stale data.

⊛ Writing Strategies (wrt Cache)

↳ Cache becomes stale if someone updates the DB (write) (CUD)

↳ I can handle WRITEs in diff. ways, each of which will impact how my DB & cache are in sync.



Write Through Cache

Write Back Cache

Write Around Cache

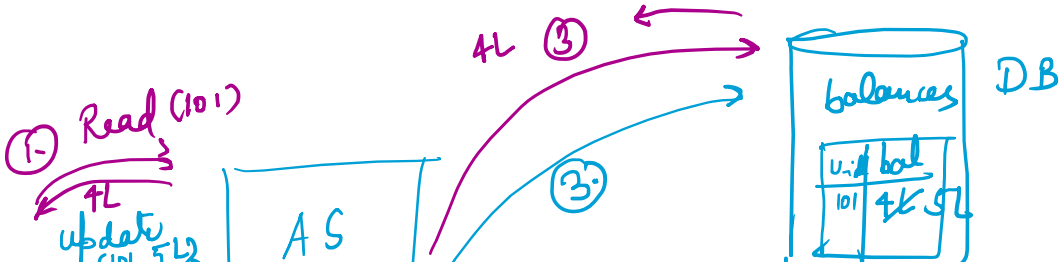
⑴ Write Through Cache :-

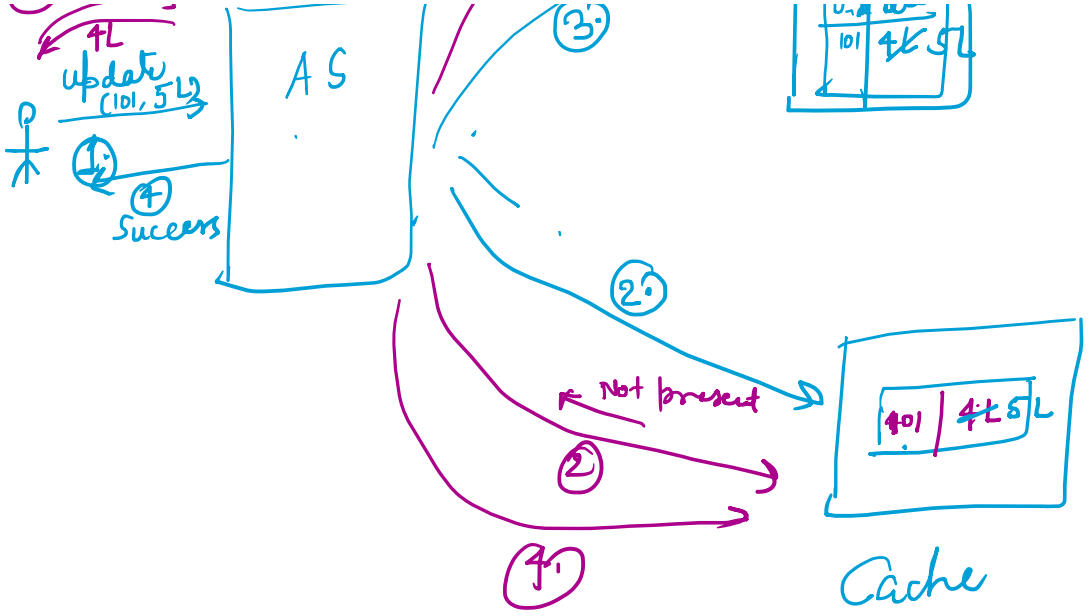
DB



App^n Server (AS) :-

1. Writes the data to Cache
2. If it fails, return Error to Cx
3. If it succeeds, write the data to DB
   - ↳ Fails :- Remove the data from Cache & Tell failure to Cx
   - ↳ Success :- Tell the Cx Success





Q.1 Will DB & Cache ever be out of sync.

Ans. NO

In other words,
if cache will have the data,
it will always have the
latest data.

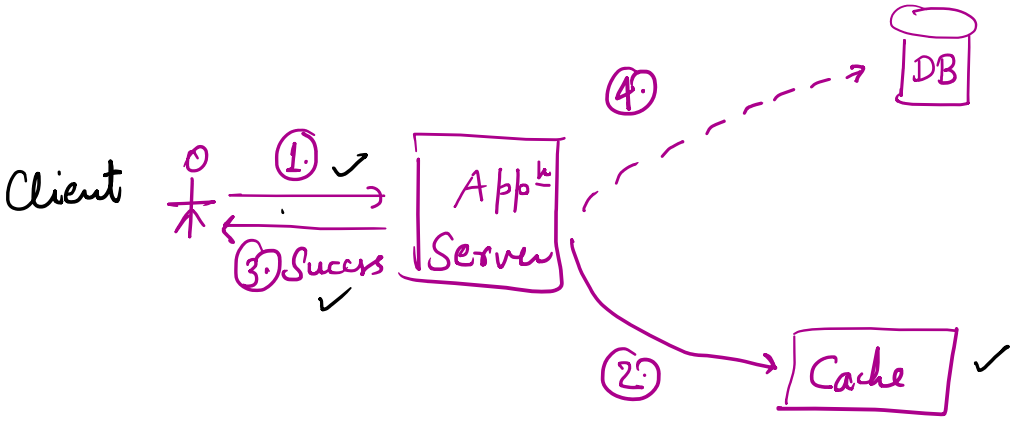
Q.2 What is the cost paid for above?
(trade-off)

↳ Write Queries will become slow.

(*) Write Back Cache

write():
↳ write data to cache
↳ Return success to client
↳ Async. try to write to DB

↳ Return success to client
↳ Async. try to write to DB



Con :- Writes may get lost forever.

[This approach works only in cases where data loss is acceptable]
↳ a little bit inconsistent data is okay.

Eg: YouTube Likes count :-

9's of availability
99%
99.9%
99.99%

⊛ Write Around Cache (TTL)




AS
DB
Cache